

Task 1: ML Classification Model

AI Internship Task Submission

1. Task Overview

This task involved building and evaluating a binary classification model to predict customer churn. A synthetic customer dataset was generated with features such as customer age, tenure, monthly charges, total charges, and support calls. Two models — Logistic Regression and Random Forest — were trained, cross-validated, and compared using standard classification metrics.

2. Environment Setup

The notebook was developed and run locally using JupyterLab. Required setup:

- Python 3 (ipykernel)
- Install dependencies: `pip install numpy pandas matplotlib seaborn scikit-learn`
- Launch JupyterLab: `jupyter lab`
- Open and run: "ML Classification Model.ipynb" from Desktop/ai_internship_tasks/

Exact command to run the notebook end-to-end from the terminal:

```
jupyter nbconvert --to notebook --execute "ML Classification Model.ipynb"
```

3. Approach / Steps Performed

- Step 1: Generated a synthetic dataset (1000 samples) with customer features and a Churn target (25% churn rate).
- Step 2: Data preprocessing — checked for missing values, split features/target, performed an 80/20 stratified train-test split, and applied StandardScaler for feature scaling.
- Step 3: Trained two models — Logistic Regression and Random Forest Classifier — using 5-fold Stratified Cross-Validation, scored on ROC-AUC.
- Step 4: Evaluated both models on the test set using Accuracy, Precision, Recall, F1-Score, and ROC-AUC, and generated a classification report for each.
- Step 5: Visualized results with an ROC curve comparison and a confusion matrix heatmap for the Random Forest model.

4. Code and Notebook Links

GitHub Repository: <https://github.com/khushimishra18112004/ai-intern-projects-internspark>

Notebook File

http://localhost:8888/notebooks/Desktop%2Fai_internship_tasks%2FML%20Classification%20Model%20task%201.ipynb

5. Results Summary

Model performance on the held-out test set (200 samples):

- Logistic Regression — Accuracy: 0.765, Precision: 0.00, Recall: 0.0000, F1-Score: 0.0000, ROC-AUC: 0.5063
- Random Forest — Accuracy: 0.745, Precision: 0.25, Recall: 0.0426, F1-Score: 0.0727, ROC-AUC: 0.5008

Both models perform close to random guessing (ROC-AUC near 0.50), which is expected since the dataset was synthetically generated with random, non-predictive features and no real relationship between the inputs and churn label. On real-world data with genuine predictive signals, the same pipeline would be expected to yield meaningfully higher performance.

6. Code Execution Screenshots

Below are the screenshots of the notebook code and its output, in execution order.



```
ML Classification Model.ipynb
File Edit View Run Kernel Tabs Settings Help

[1]: Import warnings
warnings.filterwarnings('ignore') # suppresses all warnings

Import numpy as np
Import pandas as pd
Import matplotlib.pyplot as plt
Import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, classification_report, confusion_matrix, roc_curve
)

# =====
# STEP 1: SYNTHETIC DATA GENERATION
# =====

np.random.seed(42)
n_samples = 1000

data = {
    'Customer_Age': np.random.randint(18, 70, size=n_samples),
    'Tenure_Months': np.random.randint(1, 72, size=n_samples),
    'Monthly_Charges': np.random.uniform(20, 120, size=n_samples),
    'Total_Charges': np.random.uniform(20, 5000, size=n_samples),
    'Support_Calls': np.random.randint(0, 50, size=n_samples),
    'Churn': np.random.choice([0, 1], size=n_samples, p=[0.75, 0.25]) # 25% churn rate
}
```

Screenshot 1 — Imports and Step 1: synthetic dataset generation



```
ML Classification Model.ipynb
File Edit View Run Kernel Tabs Settings Help

Support_Calls': np.random.randint(0, 50, size=n_samples),
'Churn': np.random.choice([0, 1], size=n_samples, p=[0.75, 0.25]) # 25% churn rate
}

df = pd.DataFrame(data)
print("=== Dataset Head ===")
print(df.head())

# =====
# STEP 2: DATA PREPROCESSING
# =====

print("=== Missing Values ===")
print(df.isnull().sum())

# Split Features (X) and Target (y)
X = df.drop('Churn', axis=1)
y = df['Churn']

# Train/test split (80/20) with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Screenshot 2 — Step 2: data preprocessing, train-test split, feature scaling

```
ML Classification Model.ipynb
File Edit View Run Kernel Tabs Settings Help

# =====
# STEP 3: MODEL TRAINING & CROSS-VALIDATION
# =====
log_reg = LogisticRegression(random_state=42)
rf_clf = RandomForestClassifier(random_state=42, n_estimators=100)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

log_reg_cv = cross_val_score(log_reg, X_train_scaled, y_train, cv=cv, scoring='roc_auc')
rf_cv = cross_val_score(rf_clf, X_train, y_train, cv=cv, scoring='roc_auc')

print(f"Logistic Regression Mean CV ROC-AUC: {log_reg_cv.mean():.4f}")
print(f"Random Forest Mean CV ROC-AUC: {rf_cv.mean():.4f}")

# Fit final models on full training set
log_reg.fit(X_train_scaled, y_train)
rf_clf.fit(X_train, y_train)

# =====
# STEP 4: MODEL EVALUATION & METRICS REPORT
# =====
def evaluate_model(model, X_test_data, y_true, model_name):
    y_pred = model.predict(X_test_data)
    y_prob = model.predict_proba(X_test_data)[:, 1]

    metrics = {}
    'Accuracy': accuracy_score(y_true, y_pred),
    'Precision': precision_score(y_true, y_pred),
    'Recall': recall_score(y_true, y_pred),

# =====
# STEP 5: VISUALIZATION
# =====
# Set up plotting layout
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot ROC Curves
fpr_log, tpr_log, _ = roc_curve(y_test, log_prob)
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_prob)

axes[0].plot(fpr_log, tpr_log, label=f'Logistic Regression (AUC = {log_metrics["ROC-AUC"]:.2f})')
```

Screenshot 3 — Step 3: model training and cross-validation setup

```
ML Classification Model.ipynb
File Edit View Run Kernel Tabs Settings Help

# =====
# STEP 3: MODEL TRAINING & CROSS-VALIDATION
# =====
log_reg = LogisticRegression(random_state=42)
rf_clf = RandomForestClassifier(random_state=42, n_estimators=100)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

log_reg_cv = cross_val_score(log_reg, X_train_scaled, y_train, cv=cv, scoring='roc_auc')
rf_cv = cross_val_score(rf_clf, X_train, y_train, cv=cv, scoring='roc_auc')

print(f"Logistic Regression Mean CV ROC-AUC: {log_reg_cv.mean():.4f}")
print(f"Random Forest Mean CV ROC-AUC: {rf_cv.mean():.4f}")

# Fit final models on full training set
log_reg.fit(X_train_scaled, y_train)
rf_clf.fit(X_train, y_train)

# =====
# STEP 4: MODEL EVALUATION & METRICS REPORT
# =====
def evaluate_model(model, X_test_data, y_true, model_name):
    y_pred = model.predict(X_test_data)
    y_prob = model.predict_proba(X_test_data)[:, 1]

    metrics = {}
    'Accuracy': accuracy_score(y_true, y_pred),
    'Precision': precision_score(y_true, y_pred),
    'Recall': recall_score(y_true, y_pred),

# =====
# STEP 5: VISUALIZATION
# =====
# Set up plotting layout
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot ROC Curves
fpr_log, tpr_log, _ = roc_curve(y_test, log_prob)
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_prob)

axes[0].plot(fpr_log, tpr_log, label=f'Logistic Regression (AUC = {log_metrics["ROC-AUC"]:.2f})')
```

Screenshot 4 — Step 4/5: model evaluation function and visualization setup

```

from sklearn.metrics import roc_curve, auc
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

# Plot ROC Curve (AUC = 0.81)
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf.predict(X_test))
plt.plot([0, 1], [0, 1], 'b--', label='Random Guess')
plt.plot(fpr_rf, tpr_rf, 'r-', label='Random Forest (AUC = 0.81)')
plt.title('ROC Curve (Logarithmic)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()

# Plot Random Forest Confusion Matrix as an example
cm = confusion_matrix(y_test, rf.predict(X_test))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=axes[1],
            xticklabels=['No Churn', 'Churn'], yticklabels=['No Churn', 'Churn'])
axes[1].set_title('Random Forest Confusion Matrix')
axes[1].set_xlabel('Predicted Label')
axes[1].set_ylabel('True Label')

plt.tight_layout()
plt.show()

=== Dataset Head ===
   Customer_Age  Tenure_Months  Monthly_Charges  Total_Charges  Support_Calls  \
0             56             15          59.163269         1518.998638           5
1             89             64          50.142883         1806.483317           7
2             46             18          68.587176         4828.087294           9
3             32             39          45.877657         1408.224165           3
4             60             57          48.487298         1869.338728           4

   Churn
0      0
1      0
2      0
3      0
4      0

```

Screenshot 5 — ROC curve and confusion matrix plotting code with dataset head output

```

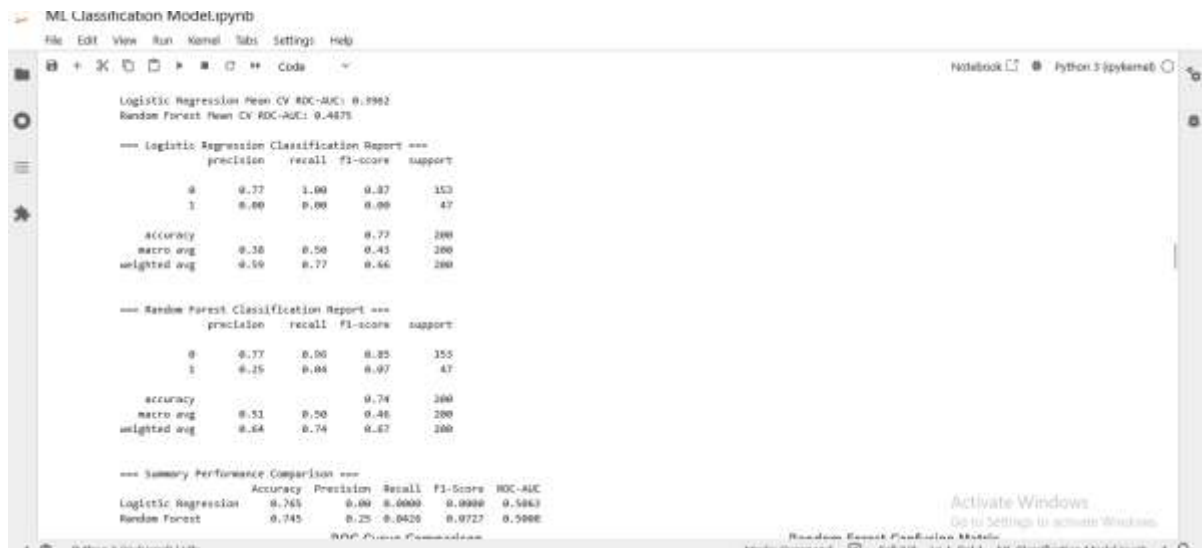
=== Missing Values ===
Customer_Age      0
Tenure_Months    0
Monthly_Charges  0
Total_Charges    0
Support_Calls    0
Churn            0
dtype: int64

Logistic Regression Mean CV ROC-AUC: 0.8963
Random Forest Mean CV ROC-AUC: 0.4878

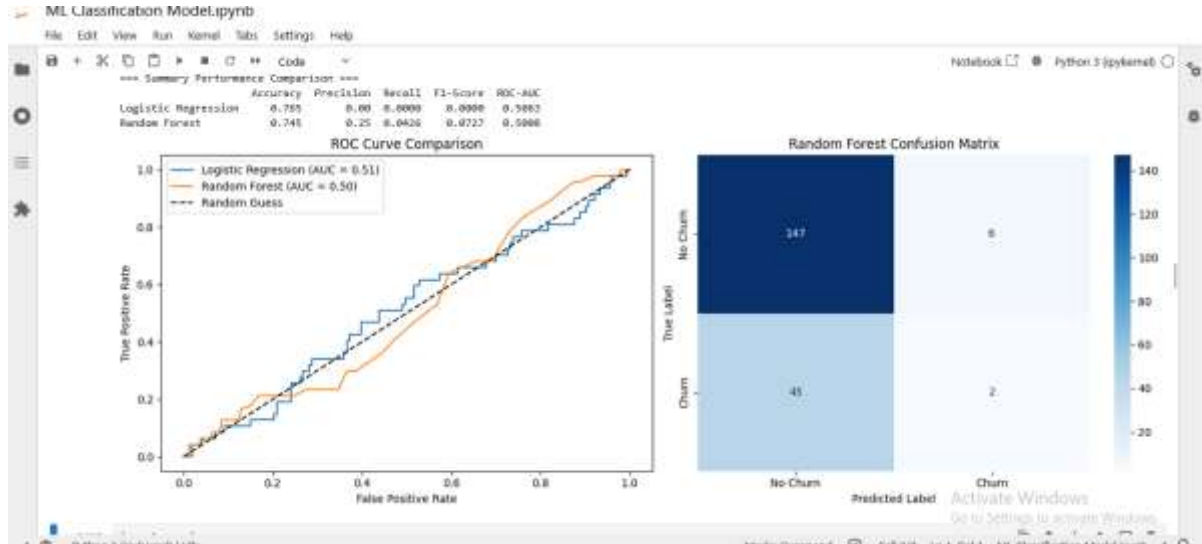
=== Logistic Regression Classification Report ===
precision    recall  f1-score   support


```

Screenshot 6 — Dataset head, missing value check, and cross-validation ROC-AUC output



Screenshot 7 — Classification reports and performance comparison table



Screenshot 8 — ROC Curve Comparison and Random Forest Confusion Matrix (final output)

End of Task 1 documentation.